useContext Hook

What is Context?

Context solves the **prop drilling** problem - passing props through multiple components that don't use them.

Core Concepts

1. State Lifting

When two sibling components need the same data, state must live in their common parent.

```
function Parent() {
  const [count, setCount] = useState(0); // State lifted here

  return (
    <>
      <ChildA count={count} setCount={setCount} />
      <ChildB count={count} />
    </>
  );
}
```

Rule: State lives in the lowest common ancestor.

---

2. Prop Drilling Problem

```
App (has state)
└── Main (doesn't use, just passes)
```

```
    └── ProductList (doesn't use, just passes)
        └── ProductCard (finally uses it!)
```

Problems:

- Intermediate components forced to know about props they don't use
- Hard to maintain and refactor
- Unnecessary re-renders of intermediate components

Context Solution

Step 1: Create Context

```
import { createContext } from 'react';

const CartContext = createContext();
export default CartContext;
```

Step 2: Provide Context (React 19+)

```
function App() {
  const [cartItems, setCartItems] = useState([]);

  return (
    <CartContext value={{ cartItems, setCartItems }}>
      <Header />
      <Main />
    </CartContext>
  );
}
```

For React 18 and below: Use `<CartContext.Provider>` instead of `<CartContext>`

Step 3: Consume Context

```
import { useContext } from 'react';
import CartContext from './CartContext';

function ProductCard({ product }) {
  const { cartItems, setCartItems } = useContext(CartContex
t);

  const addToCart = () => {
    setCartItems([...cartItems, product]);
  };

  return <button onClick={addToCart}>Add to Cart</button>;
}
```

How It Works (Mental Model)

Normal Props: Like a relay race - each component must pass to the next

Context: Like WiFi - Provider broadcasts, any component can connect directly

```
// Provider puts data in a "box"
contextBox = { cartItems, setCartItems };

// useContext reads from the "box"
const data = contextBox;
```

What Can Go in Context Value?

1. **Primitives:** `value={42}` or `value="dark"`
2. **Objects:** `value={{ theme: "dark", fontSize: 16 }}`
3. **State + Setters:** `value={{ cartItems, setCartItems }}`
4. **Functions:** `value={{ addToCart, removeFromCart }}`
5. **Computed values:** `value={{ cartItems, totalPrice }}`

Best Practices

✅ DO

```
// Wrap only the part of tree that needs context
function Dashboard() {
  return (
    <ThemeContext value="dark">
      <DashboardContent />
    </ThemeContext>
  );
}

// Use useMemo for object values
const value = useMemo(() => ({ cartItems, setCartItems }), [c
artItems]);
```

❌ DON'T

```
// Don't create new objects without memoization
<CartContext value={{ cartItems, setCartItems }}> // Re-rende
rs all consumers every time
```

Re-renders with Context

When context value changes:

- Only components using `useContext()` re-render
- Intermediate components DON'T re-render (unlike prop drilling)

```
function Parent() {
  return <Child />; // Doesn't re-render
}
```

```
function Child() {
  const { count } = useContext(CountContext); // Re-renders w
hen count changes
  return <div>{count}</div>;
}
```

Complete Example

```
// CartContext.js
import { createContext } from 'react';
const CartContext = createContext();
export default CartContext;

// App.js
import { useState, useMemo } from 'react';
import CartContext from './CartContext';

function App() {
  const [cartItems, setCartItems] = useState([]);

  const addToCart = (product) => {
    setCartItems([...cartItems, product]);
  };

  const value = useMemo(() => ({
    cartItems,
    addToCart
  }), [cartItems]);

  return (
    <CartContext value={value}>
      <Header />
      <Main />
```

```
    </CartContext>
  );
}

// ProductCard.js
import { useContext } from 'react';
import CartContext from './CartContext';

function ProductCard({ product }) {
  const { addToCart } = useContext(CartContext);

  return (
    <button onClick={() => addToCart(product)}>
      Add to Cart
    </button>
  );
}

// CartSummary.js
function CartSummary() {
  const { cartItems } = useContext(CartContext);

  return <div>Items: {cartItems.length}</div>;
}
```

Key Takeaways

1. Context solves prop drilling by creating a direct channel between Provider and consumers
2. Provider can be at any level - wrap only what needs the data
3. `useContext()` subscribes a component to context changes
4. Use `useMemo` for object values to prevent unnecessary re-renders
5. Context is not for all state - use for truly shared data (theme, auth, cart)